

Bigtable Feature Store

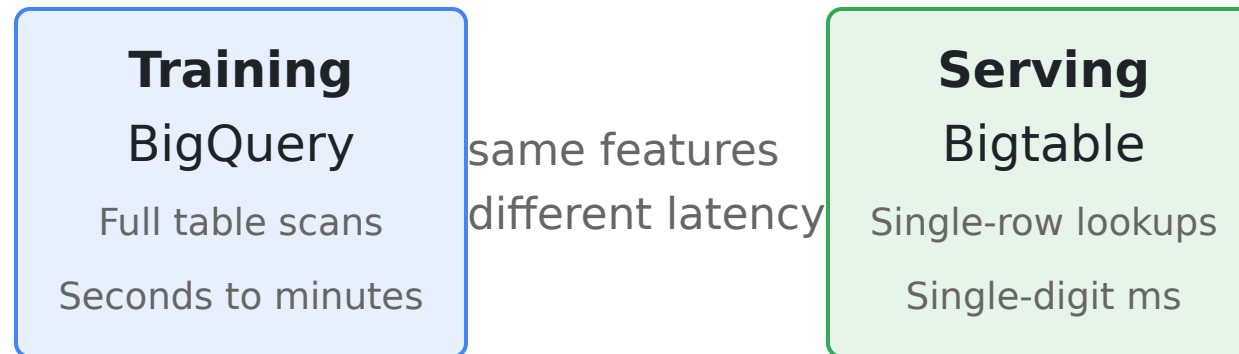
**15 notebooks from first export to production
recommendation engine**

BigQuery as the offline store | Bigtable for sub-10ms online serving

[github.com/statmike/vertex-ai-mlops/MLOps/Feature Store/Bigtable](https://github.com/statmike/vertex-ai-mlops/MLOps/Feature%20Store/Bigtable)

The Feature Store Problem

Training and **serving** have fundamentally different performance requirements.



A **feature store** bridges this gap: features are computed once in BigQuery, exported to Bigtable, and served at low latency for real-time inference.

Why Bigtable?

Core properties

- Sorted key-value store
- Single-digit ms reads by row key
- Column families for physical data grouping
- Cell versioning with automatic GC
- Scales to petabytes, billions of rows

For feature stores specifically

- Direct write path (bypass BigQuery)
- Streaming ingestion via Pub/Sub
- Multi-cluster replication (HA + geo)
- App profiles for traffic isolation
- Native `EXPORT DATA` from BigQuery

Series Overview

Foundation

NB0 · Environment
NB1 · Fundamentals
NB2 · Serialization
NB3 · Synchronization
NB4 · History & Time
Travel

Data Patterns

NB5 · Streaming & Writes
NB6 · Key Design
NB7 · Schema & Ops
NB9 · Dynamic Features
NB10 · Vectors & KNN

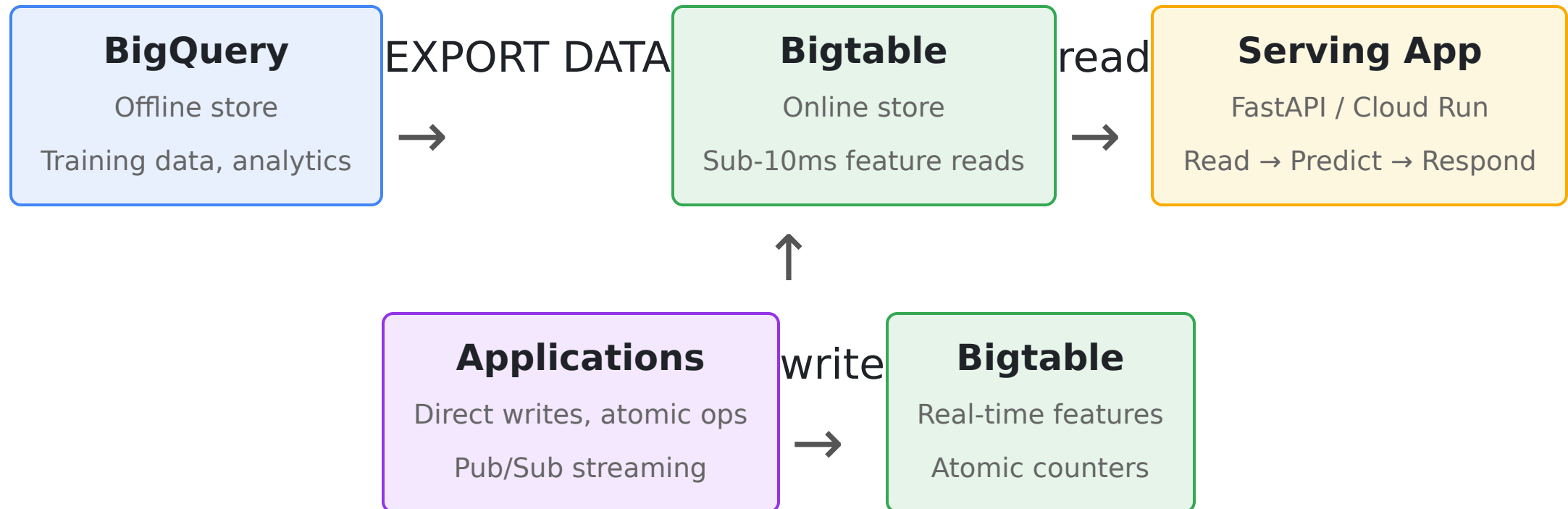
Serving

NB8 · Serving Integration
NB11 · Replication
NB12 · Emulator
NB13 · Production Deploy

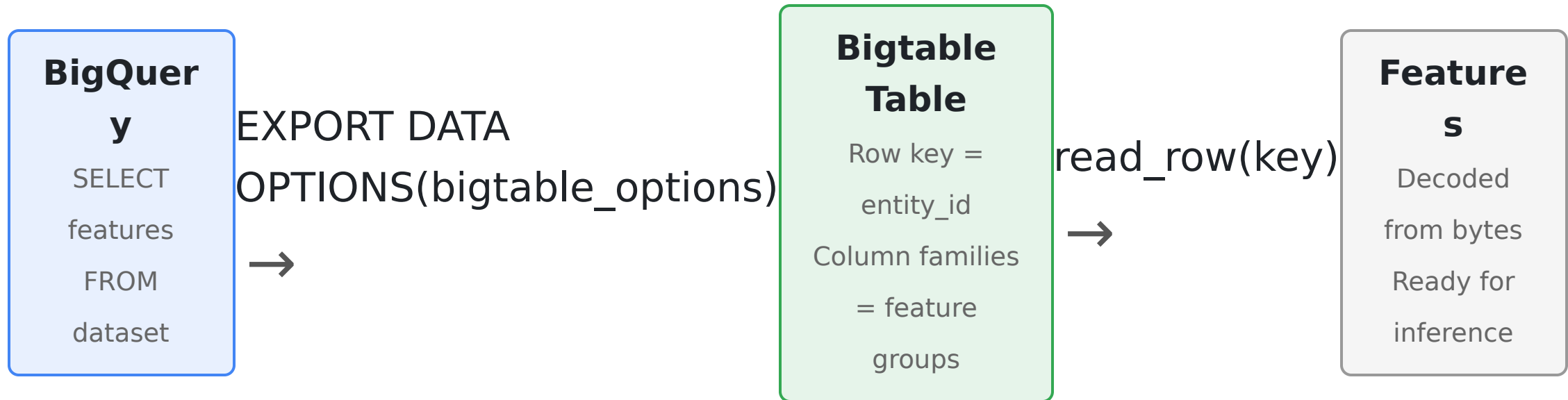
Capstone

NB14 · Recommendation
Ties all patterns together
into a complete system

Architecture



First Export: BigQuery to Bigtable



Self-describing schema: A metadata row (`#schema`) stores column names, types, and layout so clients decode without hardcoded mappings.

Data: 130,000 entities with 200+ features across every BigQuery data type (INT64, FLOAT64, STRING, ARRAY, STRUCT, JSON, GEOGRAPHY).

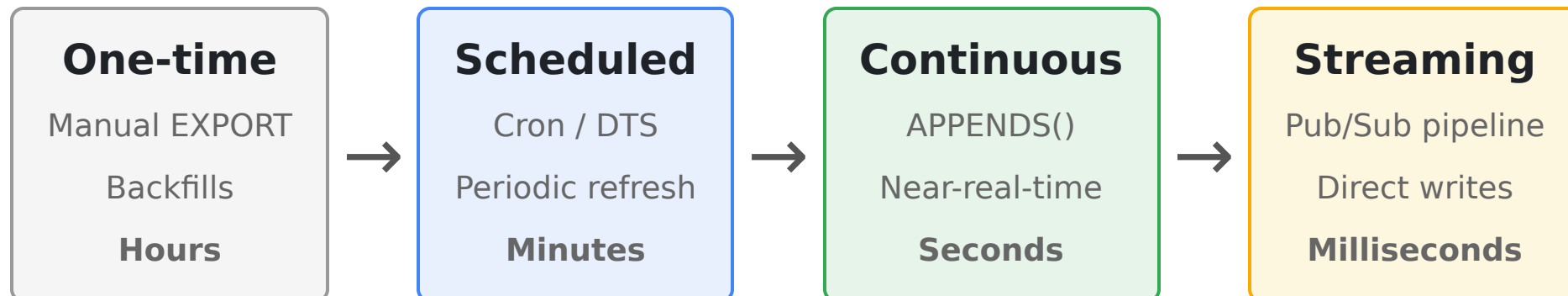
Serialization Strategies

Five methods for encoding BigQuery data into Bigtable cells:

Method	Approach	Storage	Decode Speed
Native	One BQ column → one BT qualifier	Baseline	Fast
JSON	Entire row as JSON string	Larger	Slow (parse)
Concat	Delimiter-separated values	Compact	Medium
Protobuf	Binary blob with FileDescriptorSet	Smallest	Fast
Hybrid	Native for hot features, protobuf for long tail	Balanced	Fast (hot path)

Key tradeoff: storage size vs. client-side decode CPU. Native is simplest; protobuf is most compact; hybrid optimizes for access patterns.

Keeping Data Fresh



Choose based on your freshness SLA and cost budget. Most production systems combine patterns: scheduled sync for batch features, streaming for real-time signals.

Also covered: BigQuery reservations for export jobs, Data Boost for analytics reads, continuous materialized views.

Feature History

Two strategies for storing historical feature values:

Key-based

- Timestamp embedded in row key
- `entity_id#20240115T120000`
- Range scans over time windows
- Manual garbage collection
- Best for: time-series queries

Cell versioning

- Multiple timestamped values per cell
- Automatic GC (max versions, max age)
- Cleaner API, fewer rows
- Limited query patterns
- Best for: recent history, rollback

Also: **point-in-time joins** for label-aligned training data extraction.

Direct Writes and Streaming

Bypass BigQuery for high-frequency feature updates:

Write operations

- Single-row writes
- Batch mutations (grouped RPCs)
- Atomic read-modify-write
- Check-and-mutate (conditional)



Patterns: idempotent writes, change streams, dual-write architectures

Key Design and Organization

Row key design determines performance, data locality, and query patterns.

Key patterns

- Compound: `group#entity_id`
- Namespaced: `USER#123` ,
`ITEM#456`
- Salted: `hash_prefix#entity_id`

Hotspot avoidance: no sequential keys, no timestamp-first keys, distribute writes across nodes

Column family strategy

- Group by access pattern
- Group by update frequency
- Group by GC policy
- Single-table design with multiple families

Key Visualizer: identify hot rows, spot sequential patterns, measure data distribution

Schema Evolution and Operations

Maintaining a feature store over time without downtime:

Schema management

- Versioned schemas (add, drop, rename)
- Metadata columns for decode instructions
- Feature registry (`#registry` row)
- Backfill without interrupting serving

Production operations

- Monitoring (CPU, disk, throughput)
- App profiles (serving vs. batch priority)
- Security, IAM, authorized views
- Autoscaling, cost optimization (SSD, CUDs)

Vector Storage and KNN Search

Store embedding vectors alongside scalar features and query for nearest neighbors.

Storage

- Raw binary float32 via `struct.pack()`
- Standalone column, co-located with scalar features
- Query with GoogleSQL `COSINE_DISTANCE()`

When to use Bigtable KNN

Scenario	Bigtable	Vector DB
< 100K vectors	Good	Overkill
Co-located with features	Great	Extra hop
Need ANN at scale	Linear scan	Built-in
Exact results required	Yes	Approximate

Dynamic Features

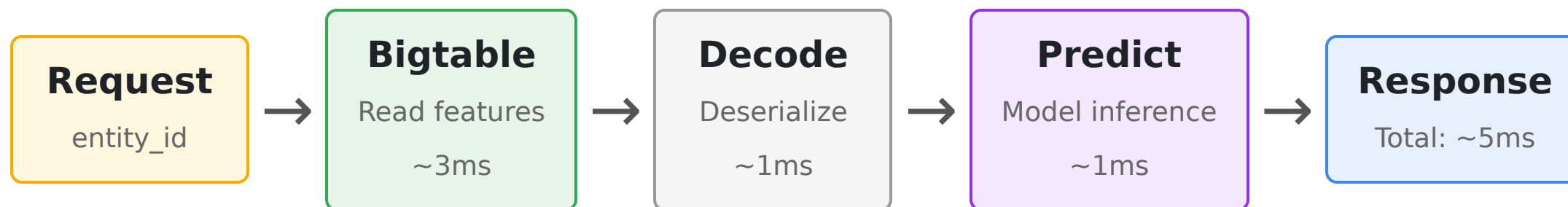
What happens when a new event arrives and the model needs features that depend on it?

Pattern	How it works	Latency	Best for
Read-Modify-Write	Atomic counter increment on each event	~5ms	Counts, sums, running totals
Read + Compute	Read stored features, compute new ones at serve time	~3ms + CPU	Ratios, time-since, combinations
Streaming Aggregation	Pub/Sub → subscriber → Bigtable	~50ms	Complex aggregations, windowed stats

Combine all three in production for a single Bigtable read.

Serving Integration

Train on BigQuery features, serve from Bigtable, measure the gap.



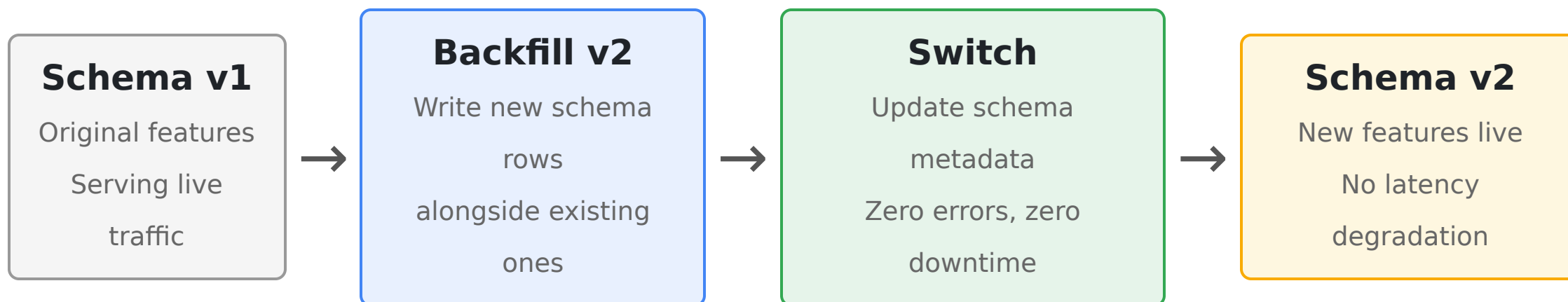
Five read methods: Python client, `cbt` CLI, GoogleSQL, BigQuery external tables, REST/gRPC

FastAPI endpoint: receive request → read features → predict → respond

Training-serving skew: compare BQ (training) vs Bigtable (serving) features to catch precision loss

Schema Evolution During Live Serving

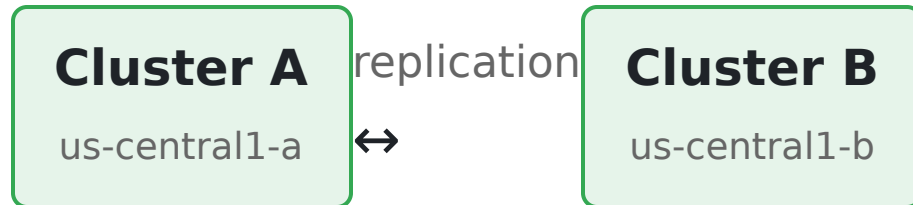
Updating the schema without taking the serving endpoint offline:



Validated with concurrent load testing: schema version switches mid-flight with no errors and no latency spikes.

Replication and High Availability

Multi-cluster replication



App profiles control routing:

- Multi-cluster → automatic failover
- Single-cluster → strong consistency

Topology options

- Single-zone (dev/test)
- Multi-zone (production HA)
- Multi-region (disaster recovery)

Measured in this series

- Replication lag between clusters
- Failover behavior
- Cost of additional clusters

Local Development with the Emulator

What the emulator provides

- `gcloud beta emulators bigtable start`
- Same Python client code
- No GCP project or billing needed
- Fast iteration for development

Testing patterns: pytest fixtures, GitHub Actions, Cloud Build, conditional test skipping

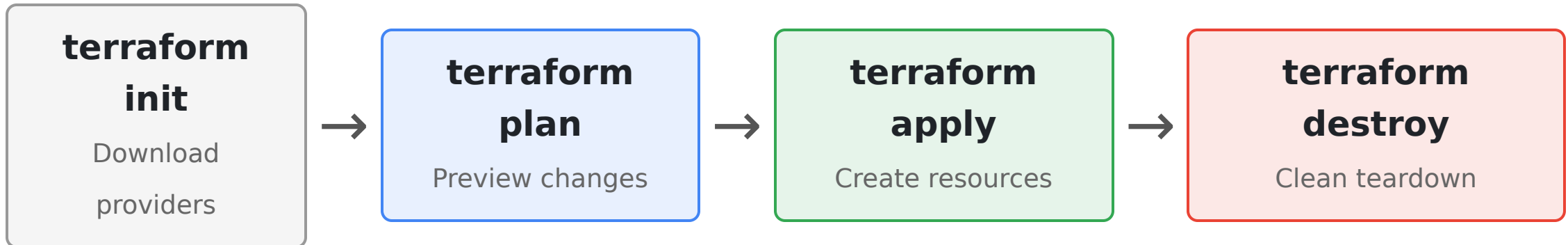
What it doesn't support

- GoogleSQL queries
- Admin API (create instance)
- Replication
- Persistence across restarts
- App profiles

Design your tests to skip GoogleSQL-dependent features when running against the emulator.

Production Deployment with Terraform

Infrastructure as Code for the complete feature store:



Managed resources: instance, clusters, tables, column families, GC policies, app profiles

Change management: add a column family in `.tf`, review the diff in `plan`, apply with confidence

Multi-Language Clients

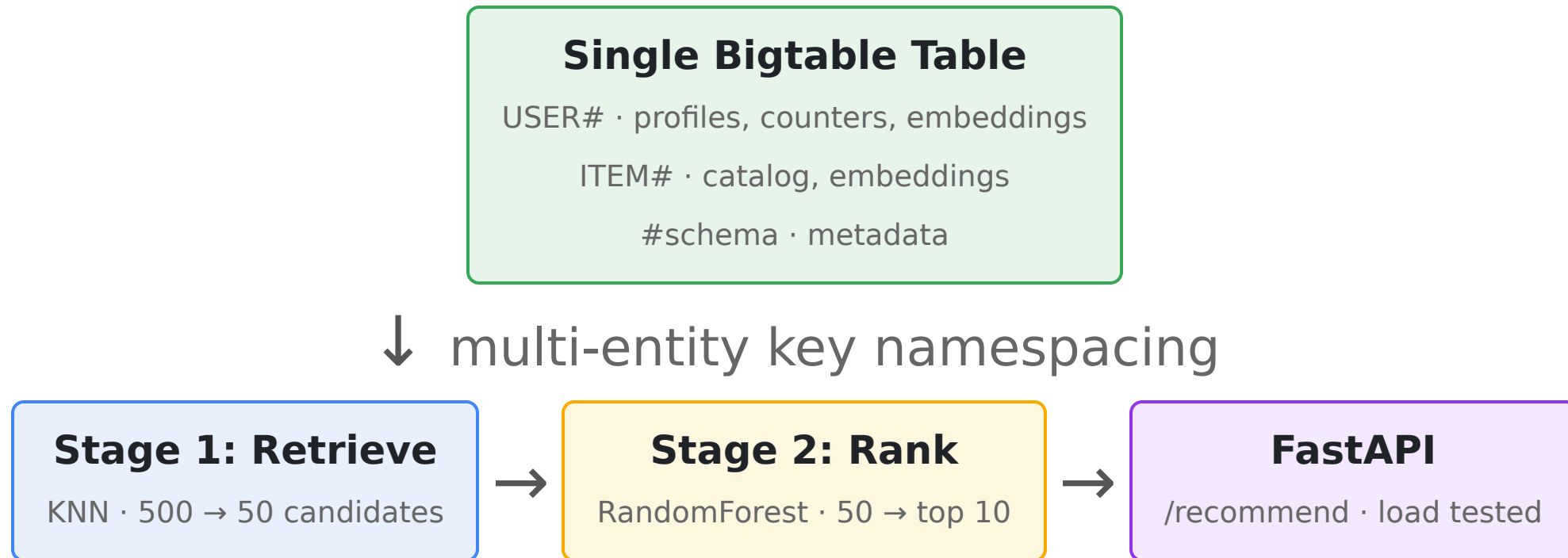
The same Bigtable feature store, accessed from different ecosystems:

Language	Strengths	Use case
Python	Data science, ML frameworks, rapid prototyping	Training pipelines, notebooks, experimentation
Go	gRPC-native, compiled, high throughput	High-performance serving endpoints
Java	Enterprise-grade, Maven ecosystem	Enterprise applications, Android backends
Node.js	Async/await, lightweight	Serverless functions, web APIs

All clients read the same rows, same column families, same byte encoding. The self-describing schema row works across languages.

Capstone: Recommendation Engine

A fully self-contained system that ties together patterns from across the entire series.



Two-Stage Recommendation Pipeline

Stage 1: Candidate Retrieval — read user embedding (64-dim), scan ITEM# rows, cosine similarity, return top-50

Stage 2: Ranking — read user profile + item catalog, compute features, RandomForest scores each, return top-10

Feature sources per prediction

Feature	Source	Update frequency
User profile	Bigtable <code>profile</code> CF	Batch (daily)
User embedding	Bigtable <code>embeddings</code> CF	Batch (daily)
Interaction counts	Bigtable <code>counters</code> CF	Real-time (atomic)
Item catalog + embedding	Bigtable <code>catalog</code> + <code>embeddings</code> CF	Batch (weekly)

Real-Time Signals and Feature Freshness

Atomic counters Key insight: combine batch + streaming

track views,
clicks, purchases.
Big-endian INT64,
no read-before-
write races.

Updated via
Pub/Sub
streaming.

Features	Quality
Stale embedding only	Misses preference shifts
Fresh counters only	No semantic understanding
Stale embedding + fresh counters	Best practical tradeoff
Fresh everything	Ideal (expensive)

Batch features (embeddings) provide semantic understanding. Real-time features (counters) provide recency. Together they outperform either alone.

Getting Started

Choose your entry point based on your experience level:

New to feature stores

Start from the beginning

NB0 → NB1 → NB2

Environment setup, first export, serialization basics

Know the basics

Jump to data patterns

NB5 → NB6 → NB9 → NB8

Streaming writes, key design, dynamic features, serving

Production-ready

Operations and deployment

NB7 → NB11 → NB13 → NB14

Schema ops, replication, Terraform, capstone

Resources

Repository: github.com/statmike/vertex-ai-mlops/MLOps/FeatureStore/Bigtable

Bigtable docs: cloud.google.com/bigtable/docs/overview

EXPORT DATA: cloud.google.com/bigquery/docs/export-to-bigtable

Vertex AI Feature Store: cloud.google.com/vertex-ai/docs/featurestore/latest/overview